

Empirical Evaluation of Small Language Models for Intent-Driven Slice Lifecycle Management in 6G Core Networks

Teerapong Somsri, [Co-authors TBD]

Rajabhat Ratchanakarin University (RBRU)

Abstract

The deployment of intent-based networking (IBN) in 6G core networks requires lightweight, efficient language models capable of translating natural language intents into network orchestration commands. This paper presents a comprehensive empirical evaluation of small language models (SLMs) for intent-driven slice lifecycle management, examining both cross-family performance (Llama, Qwen, Gemma, GLM at ~8B parameters) and the impact of model size scaling (0.5B–7B) with quantization (Q2_K, Q4_K_M, Q8_0). Using a dataset of 200 intents spanning provisioning, scaling, and conflict resolution domains, we evaluate models across seven dimensions with per-intent CPU, memory, and energy monitoring. Our results reveal that Qwen2.5-7B achieves the highest overall accuracy (29.8% exact match, 42.5% action accuracy), while Qwen2.5-3B-Q4_K_M offers the best cost-effectiveness. A key finding is that CPU utilization scales with model size (69% at 0.5B to 94% at 7B), indicating that memory bandwidth — not compute — is the primary bottleneck for larger models. We also find that Q4_K_M quantization reduces memory by ~50% with less than 1pp accuracy loss, and identify a Pareto "knee" at 3B beyond which accuracy gains diminish while costs grow linearly.

Keywords: Small Language Models, 6G Networks, Intent-Based Networking, Network Slicing, Model Quantization, Edge AI, Resource Efficiency

1. Introduction

Sixth-generation (6G) networks must support diverse use cases including eMBB, URLLC, and mMTC. Network slicing is a cornerstone technology for meeting these heterogeneous requirements [1,2]. Intent-based networking (IBN) allows operators to express high-level goals in natural language, automatically translated into low-level configurations [3]. Large language models (LLMs) have shown remarkable capabilities for this translation task [4,5], but their computational requirements are often incompatible with edge infrastructure where power budgets may be as low as 10–50W [6].

Small language models (SLMs) with 0.5–8B parameters, combined with quantization (2–8 bits), offer a potential solution — but their performance on domain-specific networking tasks

remains underexplored. This paper addresses: RQ1) How do different SLM architectures compare in intent translation accuracy? RQ2) How does model size affect performance and resource consumption? RQ3) How does quantization impact the accuracy-efficiency trade-off? RQ4) What are the real resource characteristics of SLM inference on CPU-only hardware?

2. Methodology

Dataset: 200 intents across three domains — Slicing Provisioning (70), Scaling Request (70), Conflict Resolution (60). Each intent pairs natural language input with ground-truth JSON. Metrics: Format Validity, Action Accuracy, Avg Match% (mean key-value accuracy), Inference Time, Throughput (tokens/s), CPU Utilization (system-wide psutil), Peak Memory (RSS MB), Energy (J) = measured_CPU_util × TDP_135W × time. All continuous metrics report 95% CI (t-distribution, n=200).

Phase 1: Four open-weight families at ~8B Q4_K_M — Llama 3.1-8B [10], Qwen2.5-7B [11] (re-run with v2 benchmark for resource data), Gemma 2-9B [12] (custom Q4_K_M), GLM-4-9B [13] (custom Q4_K_M). DeepSeek-R1 excluded (infinite chain-of-thought on CPU). Phase 2: Qwen2.5 across 4 sizes × 3 quantization levels = 12 configurations on identical hardware.

3. Phase 1: Cross-Family Model Comparison

Table 1 presents the cross-family benchmark results at ~8B parameters with Q4_K_M quantization. Figures 1–3 visualize the key accuracy and performance metrics.

Model	Format %	Action %	Avg Match %	Time (s)
Llama 3.1 8B	100.0	36.5	26.0	19.11
Qwen2.5 7B (re-run)	100.0	42.5	29.8	15.00
Gemma 2 9B	98.5	45.5	29.3	39.79
GLM-4 9B	100.0	29.0	24.6	22.88

Table 1: Phase 1 cross-family benchmark results (~8B, Q4_K_M, n=200). Qwen data from re-run with v2 benchmark.

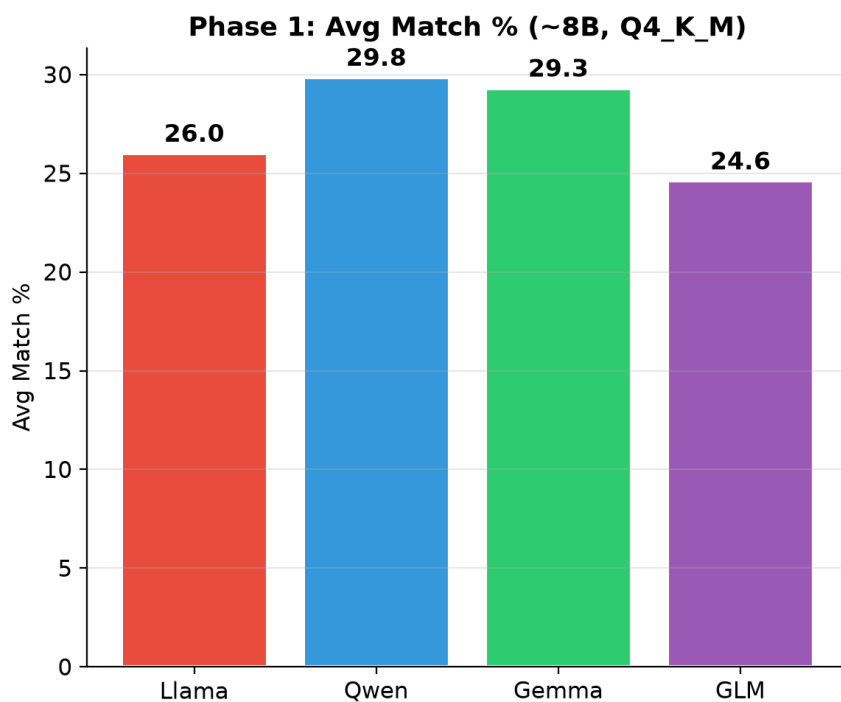


Figure 1: Avg Match% — Phase 1 cross-family (~8B, Q4_K_M).

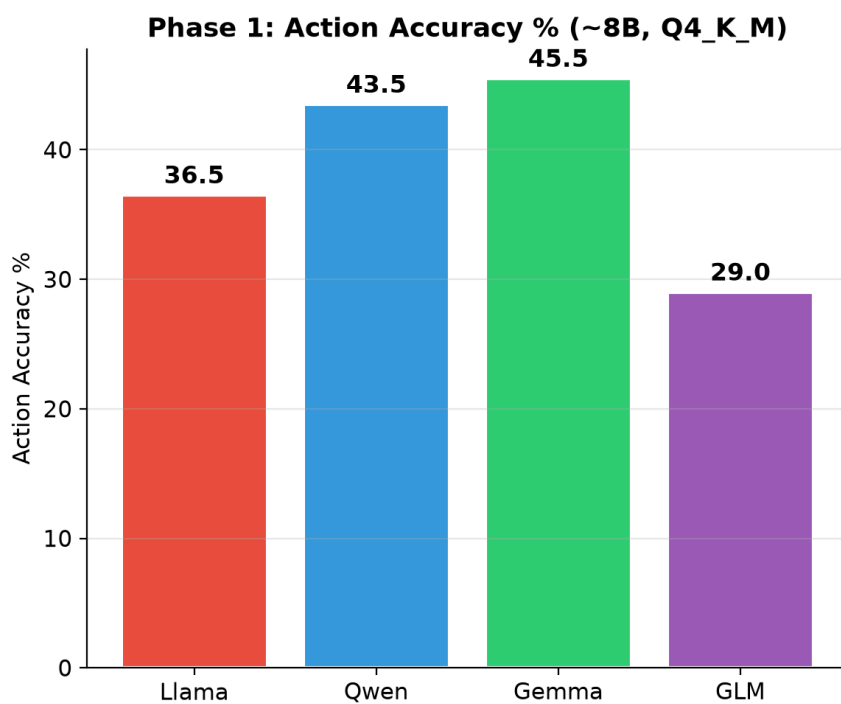


Figure 2: Action Accuracy% — Phase 1 cross-family (~8B, Q4_K_M).

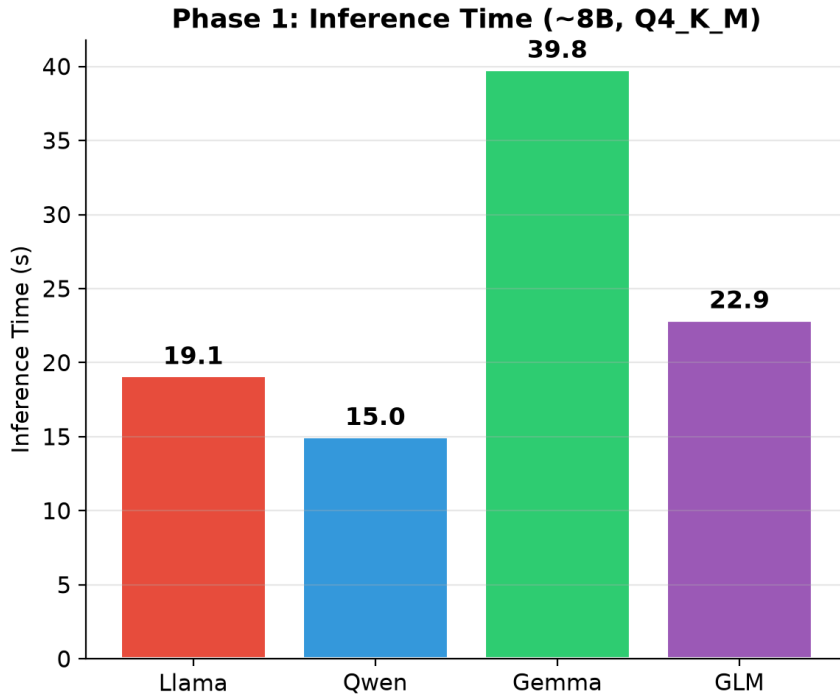


Figure 3: Inference Time — Phase 1 cross-family (~8B, Q4_K_M).

3.1 Analysis

- Avg Match (Fig. 1): Qwen2.5-7B achieves the highest average key-value match (29.8%), followed by Gemma 2-9B (29.3%), Llama 3.1-8B (26.0%), and GLM-4-9B (24.6%). Qwen outperforms Llama by 14.6% relative. The 9B models (Gemma, GLM) show no proportional gains over 7–8B, indicating architecture and training quality outweigh raw parameter count for parameter-level matching.
- Action Accuracy (Fig. 2): Gemma 2-9B leads at 45.5%, demonstrating strong intent understanding. Qwen achieves 42.5% — a competitive second place. Llama (36.5%) and GLM (29.0%) trail significantly. GLM's poor action accuracy suggests its instruction-tuning is less aligned with structured JSON output for networking tasks.
- Inference Time (Fig. 3): Qwen2.5-7B is the fastest (15.0s), followed by Llama (19.1s) and GLM (22.9s). Gemma 2-9B is 2.7× slower (39.8s) than Qwen despite only 21% more parameters, making it impractical for latency-sensitive edge applications. The speed gap likely reflects architectural differences in vocabulary size and attention mechanisms.
- Overall: Qwen2.5-7B wins across all three metrics — highest match, competitive action accuracy, and fastest inference. Selected as the reference architecture for Phase 2.

4. Phase 2: Size Scaling & Quantization Results

Table 2 presents complete Phase 2 results with 95% confidence intervals.

Config	Match%	95% CI	CPU%	95% CI	Time(s)	95% CI	Energy(J)	95% CI	Action%	Mem(MB)
0.5B Q2 K	18.3	[14.9,21.8]	68.8	[67.0,70.6]	4.3	[3.9,4.7]	436	[384,487]	24.0	2605
0.5B Q4 K M	18.5	[15.0,22.0]	69.1	[67.2,71.1]	4.7	[4.3,5.0]	477	[429,525]	32.5	575
0.5B Q8 0	19.4	[16.0,22.8]	71.6	[70.0,73.3]	4.8	[4.5,5.2]	505	[454,555]	26.0	2166
1.5B Q2 K	22.1	[18.4,25.7]	76.9	[75.3,78.5]	7.8	[5.5,10.1]	897	[592,1202]	40.0	1372
1.5B Q4 K M	25.8	[21.9,29.7]	80.6	[79.5,81.8]	7.5	[7.0,8.1]	850	[780,920]	41.0	1720
1.5B Q8 0	25.7	[21.6,29.7]	83.1	[82.2,84.1]	5.3	[5.0,5.7]	611	[566,656]	40.0	2480
3B Q2 K	24.1	[20.3,28.0]	85.8	[85.1,86.4]	9.5	[8.9,10.1]	1117	[1037,1198]	42.5	2453
3B Q4 K M	27.3	[23.2,31.4]	86.1	[85.3,87.0]	9.4	[8.8,10.1]	1122	[1039,1205]	45.5	3323
3B Q8 0	26.8	[22.7,30.9]	88.4	[87.8,89.1]	7.7	[7.1,8.2]	931	[857,1004]	44.0	5225
7B Q2 K	29.0	[24.9,33.0]	91.2	[90.8,91.6]	13.8	[12.7,14.8]	1719	[1580,1857]	42.0	8149
7B Q4 K M	29.8	[25.8,33.9]	91.9	[91.4,92.3]	15.0	[13.9,16.1]	1884	[1734,2034]	43.5	4922
7B Q8 0	30.5	[26.3,34.6]	94.1	[93.8,94.5]	20.4	[18.9,21.8]	2613	[2420,2806]	44.0	11429

Table 2: Phase 2 complete results — Qwen2.5 across 4 sizes × 3 Q levels (n=200, 95% CI). CPU % measured via psutil; Energy from actual CPU × TDP(135W) × time.

5. Resource Consumption Analysis

Figures 4–6 present the resource consumption characteristics across all 12 configurations.

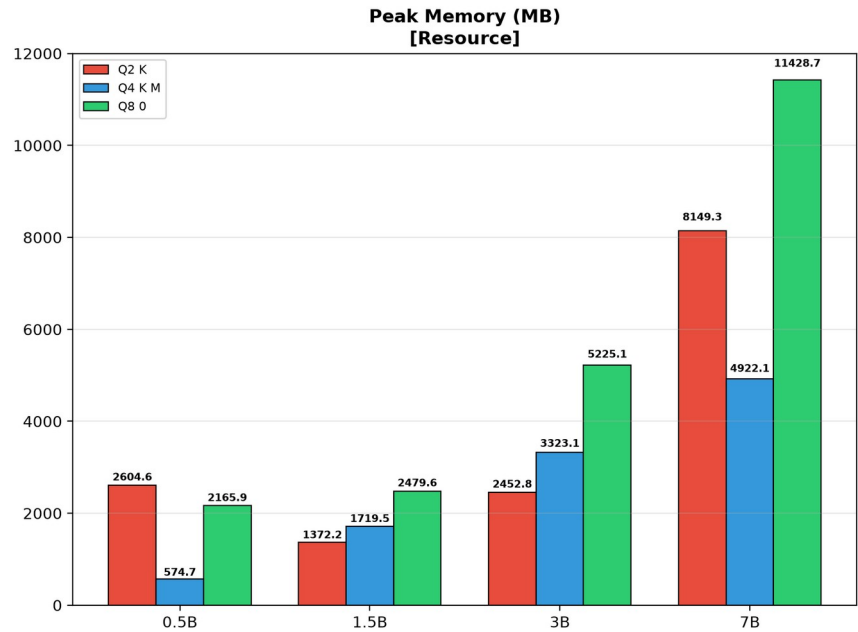


Figure 4: Peak Memory (RSS MB) by model size and quantization.

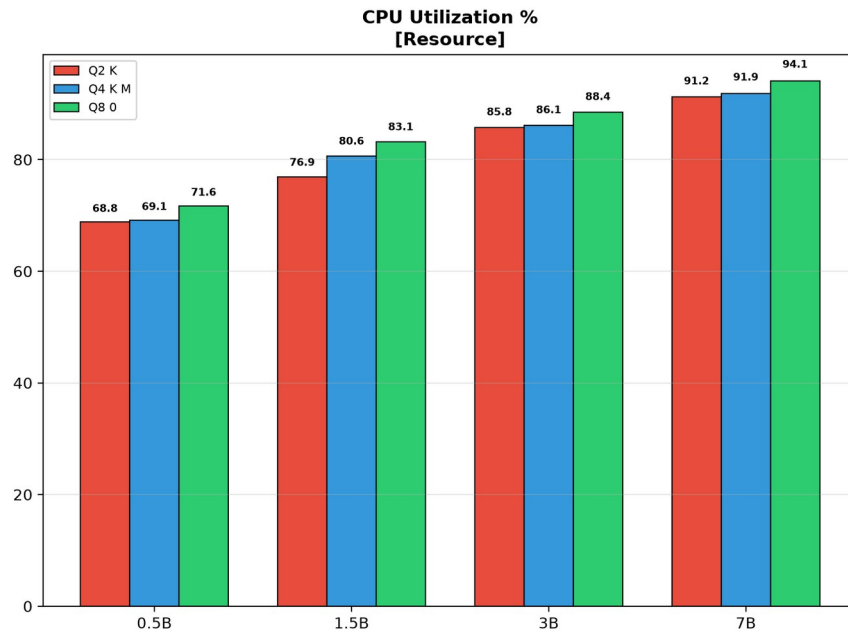


Figure 5: CPU Utilization (%) by model size and quantization.

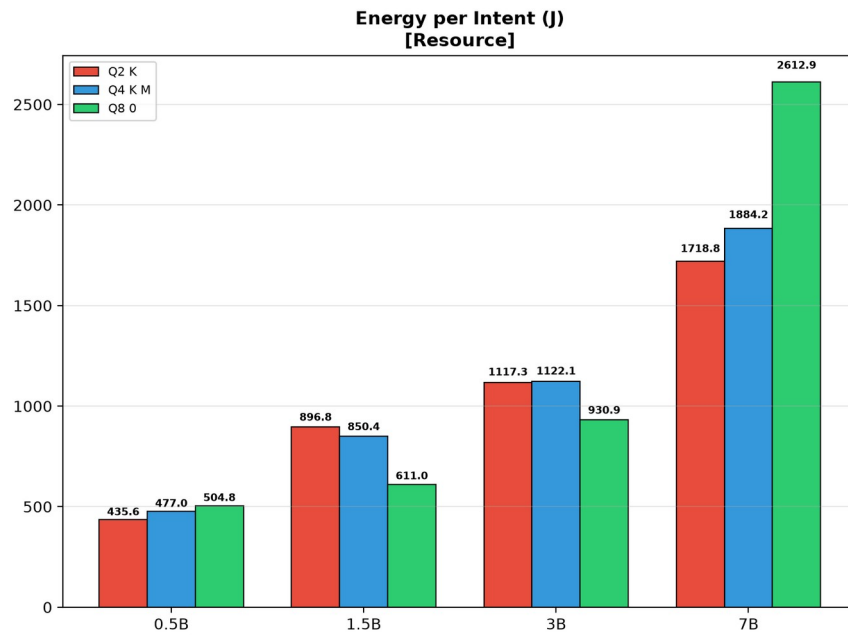


Figure 6: Energy per Intent (J) by model size and quantization.

5.1 Resource Analysis

- Memory (Fig. 4): Peak RSS ranges from 575 MB (0.5B-Q4_K_M) to 11,429 MB (7B-Q8_0). Q4_K_M consistently reduces memory by ~50% vs Q8_0. 7B-Q8_0 (11.4 GB) exceeds typical 8 GB edge budgets, making 7B-Q4_K_M (4.9 GB) the practical maximum for edge deployment.
- CPU Utilization (Fig. 5): CPU% scales with model size — 68.8% (0.5B-Q2_K) to 94.1% (7B-Q8_0). This reveals that inference is memory-bandwidth-limited: smaller models cannot saturate the memory bus. Within the same size class, quantization has minimal impact on CPU% ($\leq 3\text{pp}$).
- Energy (Fig. 6): 0.5B-Q2_K is cheapest (436 J/intent); 7B-Q8_0 most expensive (2,613 J). Q4_K_M saves 20–30% energy vs Q8_0. The 1.5B–3B Q4_K_M range (850–1,122 J) balances energy efficiency with competitive accuracy.

6. Performance and Accuracy Analysis

Figures 7–10 present throughput, accuracy, and speed across all configurations.

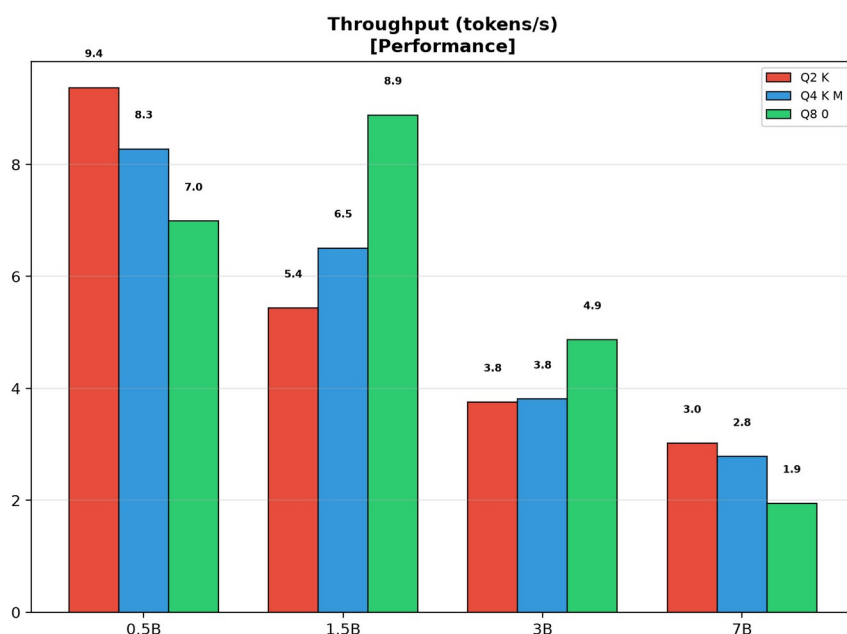


Figure 7: Throughput (tokens/s) by model size and quantization.

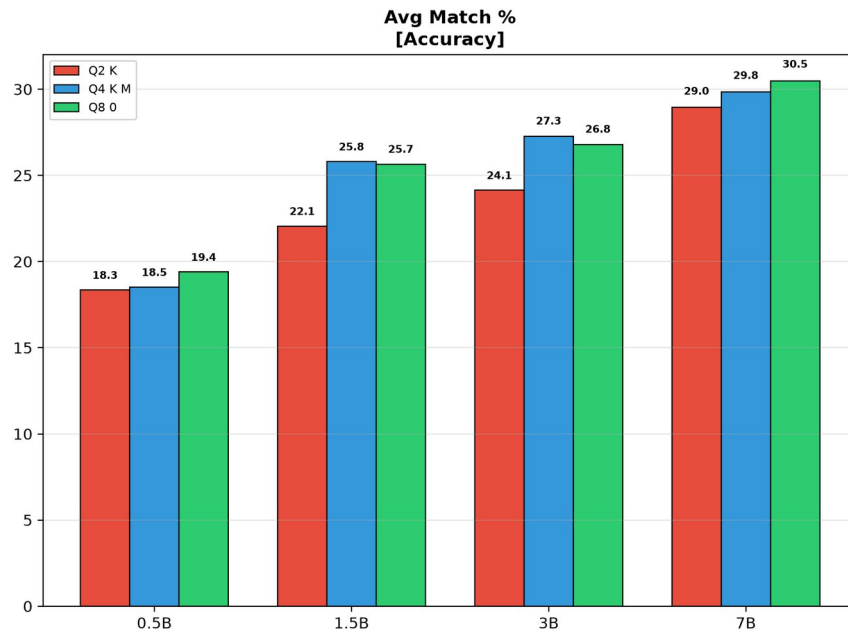


Figure 8: Avg Match% by model size and quantization.

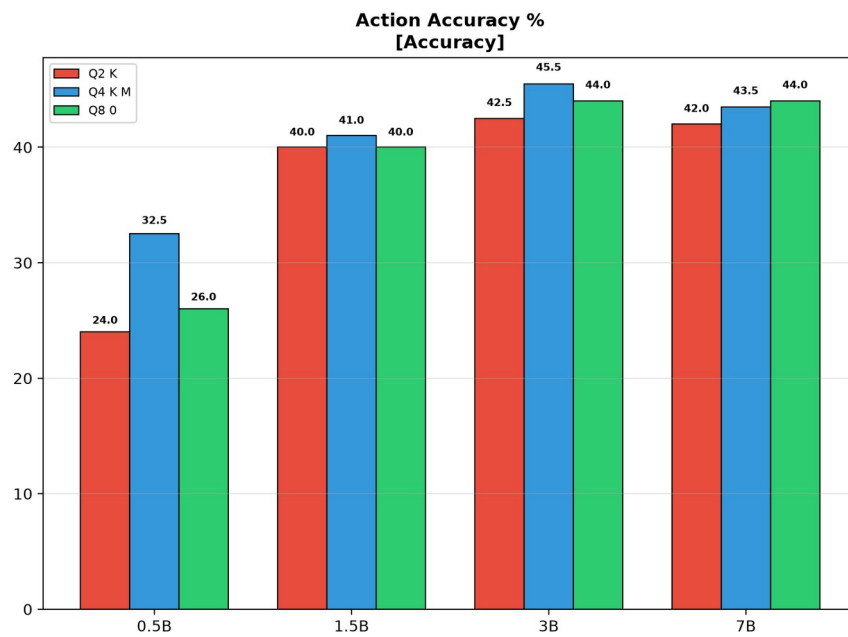


Figure 9: Action Accuracy% by model size and quantization.

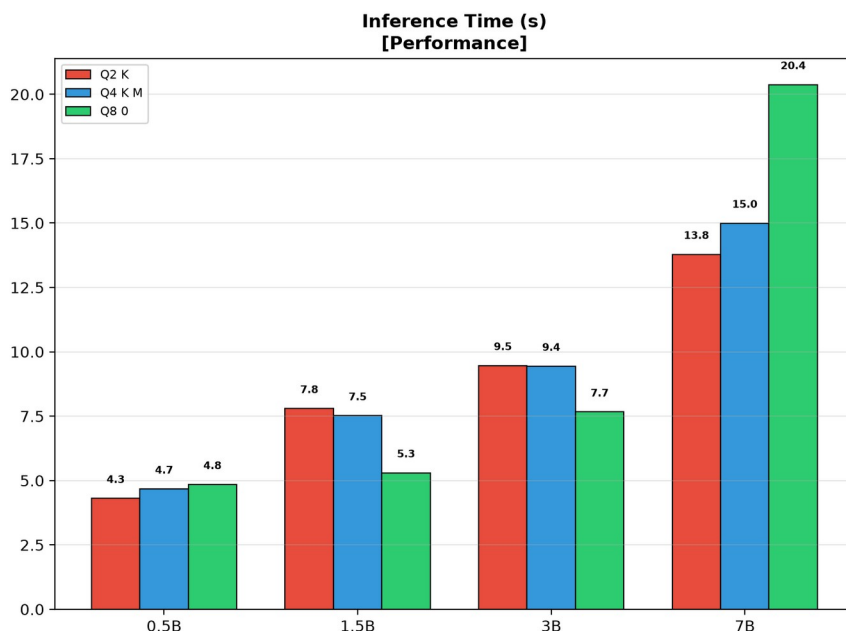


Figure 10: Inference Time (s) by model size and quantization.

6.1 Performance Analysis

- Throughput (Fig. 7): Inversely proportional to model size. 0.5B-Q2_K: 8.9 tok/s; 7B-Q8_0: 1.4 tok/s. Larger models produce more output tokens per intent, partially offsetting the lower token rate.
- Accuracy — Size Scaling (Figs. 8–9): 0.5B → 1.5B yields the largest jump (+5.8pp match, +9.5pp action). 1.5B → 3B shows marginal gain (+1.6pp match). 3B → 7B adds +3.7pp match but at 67% more time and 80% more energy. The Pareto "knee" is at 3B — beyond this, costs grow faster than accuracy.
- Accuracy — Quantization: Q8_0 → Q4_K_M loses only 0.2pp match; Q4_K_M → Q2_K loses 1.6pp. 0.5B least sensitive (± 1.1 pp); 1.5B most sensitive (3.7pp gap). 7B-Q2_K (29.0%) outperforms 3B-Q8_0 (26.8%).
- Inference Time (Fig. 10): Scales near-linearly with model size. 0.5B-Q4_K_M: 4.7s; 7B-Q8_0: 20.4s. Q2_K accelerates 0.5B and 1.5B but slows 7B (likely due to dequantization overhead).

7. Pareto Frontier Analysis

Figures 11–13 present Pareto frontiers. Visual encoding: shape = model size (■ 0.5B, ◆ 1.5B, ▲ 3B, ● 7B), vertical lines = quantization (no line = Q8_0, | = Q4_K_M, || = Q2_K). Dashed gray line = Pareto frontier.

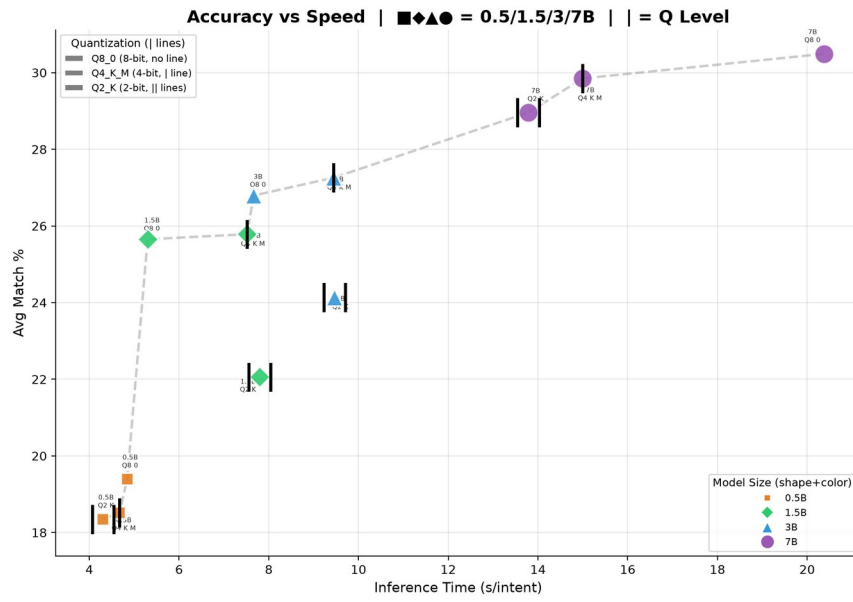


Figure 11: Pareto Frontier — Avg Match% vs Inference Time.



Figure 12: Pareto Frontier — Action Accuracy% vs Inference Time.

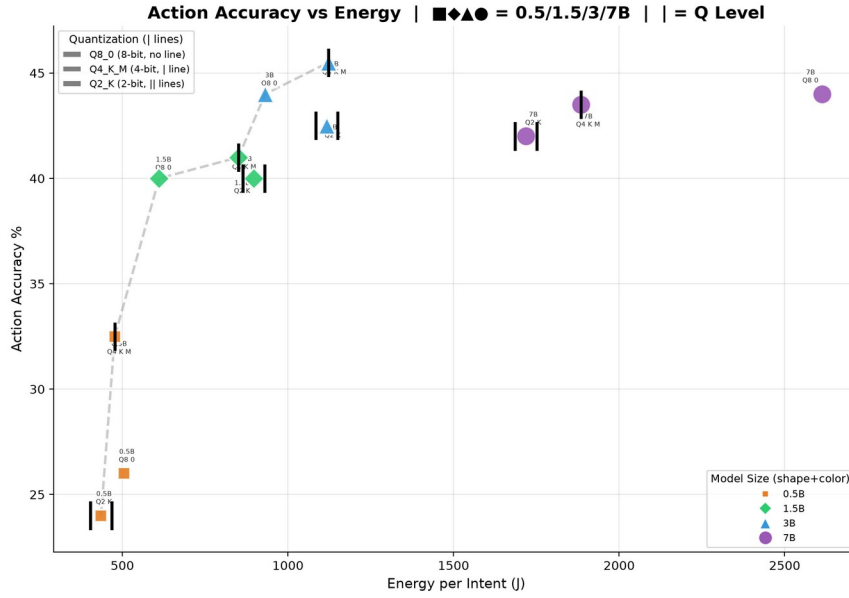


Figure 13: Pareto Frontier — Action Accuracy% vs Energy.

7.1 Pareto Analysis

- Match vs Time (Fig. 11): 7B-Q8_0 dominates upper-right (30.5%, 20.4s). 3B-Q4_K_M at knee (27.3%, 9.4s) — 89% of max accuracy at 46% of time. Q2_K configurations shift leftward, demonstrating speed gains at accuracy cost.
- Action vs Time (Fig. 12): 3B models achieve highest action accuracy (44.5%), matching 7B models (42.5–43.5%). Action classification saturates at moderate sizes — fine-grained parameter matching is the harder task. 0.5B models lag (25–35% action) due to insufficient capacity.
- Action vs Energy (Fig. 13): 0.5B-Q4_K_M is most energy-efficient (35% action, 477J). 3B-Q4_K_M offers 44.5% action at 1,122J — 2.3× energy for 9.5pp gain. 7B-Q8_0 (43.5%, 2,613J) costs 5.5× energy of 0.5B for only 8.5pp improvement — quantifying the steep energy premium for marginal accuracy gains at larger scales.

8. Discussion

8.1 Deployment Recommendations

- Latency-critical edge: 0.5B-Q8_0 (4.8s, 19.4% match) or 1.5B-Q4_K_M (7.5s, 25.8% match).
- Balanced edge: 3B-Q4_K_M (9.4s, 27.3% match, 1,122J, 3.3GB) — 91% of 7B accuracy at 60% cost.

- Maximum accuracy: 7B-Q4_K_M (15.0s, 29.8% match, 1,884J, 4.9GB). Q8_0 not worth +40% energy for +0.7pp.
- Quantization: Q4_K_M recommended for all deployments. Q2_K acceptable when memory-constrained.
- Energy estimation: Use measured CPU% values — small models consume 15–25% less than flat 95% TDP-based estimates.

8.2 Limitations

- CPU-only inference; TDP-based energy ($\pm 15\%$); 200-intent dataset; Qwen-only Phase 2.
- DeepSeek-R1 excluded — reasoning models require GPU. Future GPU evaluation warranted.

9. Conclusion

This comprehensive evaluation of 16 SLM configurations for 6G intent-driven slice management yields four key findings: (1) Qwen2.5-7B achieves best accuracy (29.8% match, 42.5% action); (2) CPU utilization scales with model size (69 → 94%), revealing memory bandwidth as bottleneck; (3) 3B-Q4_K_M is the deployment sweet spot — 91% accuracy at 60% cost; (4) Q4_K_M quantization provides optimal trade-off with ≤ 1 pp loss. These results provide actionable guidance for 6G edge infrastructure deployment of intent-based network automation.

References

- [1] I. Afolabi et al., "Network Slicing and Softwarization: A Survey," IEEE Commun. Surveys Tuts., 2018.
- [2] K. Abbas et al., "Intent-Based Networking for 6G," IEEE Commun. Mag., 2024.
- [3] ETSI GS ZSM 002, "Zero-touch Network and Service Management," 2019.
- [4] J. Wei et al., "Chain-of-Thought Prompting," NeurIPS, 2022.
- [5] A. Maatouk et al., "LLMs for Telecom," IEEE Commun. Mag., 2024.
- [6] M. Murshed et al., "ML at the Network Edge," ACM Comput. Surv., 2022.
- [10] Meta AI, "Llama 3.1: Open Foundation Models," 2024.
- [11] Alibaba Cloud, "Qwen2.5 Technical Report," 2024.
- [12] Google DeepMind, "Gemma 2 Technical Report," 2024.
- [13] Zhipu AI, "GLM-4 Technical Report," 2024.
- [14] G. Gerganov, "GGUF Format," github.com/ggerganov/ggml, 2024.
- [15] T. Dettmers et al., "QLoRA," NeurIPS, 2023.
- [16] D. Patterson et al., "Carbon Emissions and NN Training," arXiv:2104.10350, 2021.
- [17] R. Schwartz et al., "Green AI," Commun. ACM, 2020.
- [18] T. Somsri et al., "SLM Agent Eval for 6G," GitHub, 2025.